CSAPP 9.9 읽기 전 선행 개념 정리

이 문서는 재혁님이 **9.9 Dynamic Memory Allocation**을 읽기 전에, 먼저 머릿속에 넣어두면 좋은 개념만 압축한 사전 가이드입니다. 책 순서를 크게 벗어나지 않고, 실제 `malloc-lab`과 연결되는 쪽으로 정리했습니다.

heap | block | alignment | fragmentation | implicit free list | split | coalesce | boundary tag

먼저 알고 읽으면 좋은 한 줄

9.9는 단순히 `malloc` 함수 설명만 하는 장이 아니라, **운영체제가 준 힙 메모리를 allocator가 어떻게 block 단위로 관리하는지**를 설명하는 파트입니다.

1. 9.9에서 제일 먼저 잡아야 하는 큰 그림

프로그램이 실행 중 추가 메모리가 필요하다 → allocator가 heap에서 적당한 block을 찾는다 → 없으면 heap을 더 늘린다 → block을 나눠 주거나 합친다 → 나중에 free되면 다시 재사용한다.

- **커널**은 큰 틀에서 힙 영역을 제공하고,
- **allocator**는 그 힙을 잘게 block으로 나눠서 관리합니다.
- 재혁님이 이번 주에 보는 `malloc-lab`은 이 allocator를 직접 만들어보는 과제입니다.

2. heap, block, payload를 먼저 구분하기

heap

프로그램이 실행 중 동적으로 커질 수 있는 메모리 영역입니다. 보통 초기 데이터 영역 뒤에서 시작해서 높은 주소 방향으로 자랍니다.

block

allocator가 heap을 관리하기 위해 나눈 한 덩어리입니다. 어떤 block은 사용 중이고, 어떤 block은 비어 있습니다.

payload

사용자가 실제로 쓰는 데이터 부분입니다. 하지만 block 안에는 payload 말고도 관리용 정보인 **header**, 경우에 따라 **footer**가 같이 들어갑니다.

block을 아주 단순하게 보면 이런 느낌입니다

```
[ header | payload | footer ]

header: block 크기, 할당 여부 같은 관리 정보
payload: 사용자가 실제로 쓰는 공간
footer: 뒤쪽에서 이전 block 정보를 보기 위한 보조 정보
```

3. 왜 header와 정렬(alignment)이 필요한가

개념	왜 필요한가
header	이 block의 크기가 얼마인지, 현재 사용 중인지 free인지 알아야 다음 block으로 이동하거나 합칠 수 있습니다.
alignment	반환 주소가 8 또는 16의 배수가 되도록 맞춰야 여러 자료형을 안전하고 빠르게 저장할 수 있습니다.
minimum block size	header/footer와 정렬 조건 때문에 실제 요청 크기보다 block이 더 크게 잡힐 수 있습니다.

책에서도 malloc이 단순히 요청한 바이트 수만 딱 주는 게 아니라, 정렬과 관리 정보를 반영한 block을 준다고 설명합니다.

4. malloc, free, realloc을 개념적으로 보기

함수	개념적으로 하는 일
`malloc(size)`	size를 담을 수 있는 free block을 찾아 주고, 필요하면 block을 쪼개서 일부를 할당합니다.
`free(ptr)`	그 block을 다시 free 상태로 바꾸고, 주변 free block과 합칠 수 있으면 합칩니다.
`realloc(ptr, size)`	기존 block 크기를 바꾸는 함수입니다. 늘리거나 줄이면서 필요하면 새 block을 잡고 데이터를 옮길 수도 있습니다.

중요한 주의점

`free`는 **malloc/calloc/realloc**으로 받은 block 시작 주소에만 써야 합니다. 잘못된 주소를 넘기면 동작이 정의되지 않습니다.

5. 왜 동적 메모리 할당이 필요한가

- 실행 전에 데이터 크기를 모를 때가 많습니다.
- 연결리스트, 트리, 가변 크기 버퍼처럼 실행 중 크기가 바뀌는 구조가 많습니다.
- 정적 배열만으로는 메모리를 낭비하거나 한계를 정해버리게 됩니다.

책의 예시도 결국 **"실행되기 전에는 크기를 모르는 데이터를 어떻게 저장할까?"**에서 출발합니다. 이 질문이 납득되면 왜 allocator가 필요한지 훨씬 잘 이해됩니다.

6. 단편화(fragmentation)는 두 종류로 나뉩니다

종류	뜻	예시 감각
internal fragmentation	할당된 block이 payload보다 커서 block 내부에 남는 낭비입니다.	13바이트만 필요했는데 정렬 때문에 24바이트 block을 줘서 안쪽에 남는 공간
external fragmentation	free 공간 총합은 충분하지만 한 덩어리로 크게 붙어 있지 않아 요청을 못 처리하는 경우입니다.	4칸짜리 free block 두 개는 있는데 8칸 연속 공간이 없는 상태

재혁님은 여기서 특히 **external fragmentation 때문에 coalesce가 왜 필요한지**를 연결해서 보면 좋습니다.

7. implicit free list는 무엇인가

9.9에서 본격적으로 allocator 구조가 시작될 때, 책은 가장 단순한 방식인 **implicit free list**부터 설명합니다.

- free block들만 따로 리스트 포인터로 연결하지 않습니다.
- 각 block header의 **size** 정보를 이용해서 다음 block으로 이동합니다.
- 즉, heap 전체를 처음부터 끝까지 훑으면서 free block을 찾는 방식입니다.

implicit free list의 머릿속 그림

```
[16/a][24/f][32/a][48/f][0/a]

size를 읽고 다음 block으로 이동
a = allocated, f = free
마지막의 0/a 는 epilogue 같은 끝 표시 역할
```

장점은 단순함이고, 단점은 탐색이 heap 전체 길이에 비례해서 느릴 수 있다는 점입니다.

8. free block을 찾는 방법: placement policy

정책	아이디어	느낌
first fit	처음 만나는 충분히 큰 free block을 바로 사용	빠르고 단순해서 lab 초반 구현에 적합
next fit	지난 탐색 위치부터 이어서 찾음	first fit의 반복 스캔을 조금 줄이려는 시도
best fit	가장 딱 맞는 free block을 찾음	탐색 비용이 커질 수 있음

9. split과 coalesce는 allocator의 핵심 동작입니다

split

큰 free block을 찾았을 때, 요청 크기만큼만 떼어 주고 남는 부분은 다시 free block으로 남기는 것입니다.

coalesce

free된 block 양옆에 free block이 있으면 하나로 합쳐서 더 큰 free block을 만드는 것입니다.

- **split**은 internal fragmentation을 줄이는 데 도움을 줍니다.
- **coalesce**는 external fragmentation을 줄이는 데 중요합니다.

- 책은 **immediate coalescing**과 **deferred coalescing**을 구분해서 설명합니다.

왜 immediate coalescing만이 정답은 아닌가

바로 합쳤다가 곧 다시 쪼개는 패턴이 반복되면 오히려 비효율적일 수 있습니다. 그래서 실제 allocator는 나중에 한꺼번에 합치는 전략도 씁니다.

10. boundary tag는 왜 등장하는가

다음 block은 현재 block의 size를 알면 쉽게 찾을 수 있습니다. 문제는 **이전 block**입니다. 이전 block을 빨리 찾으려면 뒤쪽에도 정보가 필요합니다.

boundary tag는 block 끝에 footer를 둬서, 이전 block의 크기와 상태를 빠르게 확인하게 해주는 기법입니다.

- header만 있으면 이전 block을 찾기 어렵습니다.
- footer까지 있으면 이전 block의 끝에서 정보를 읽어 바로 합칠 수 있습니다.
- 덕분에 coalesce를 상수 시간에 가깝게 처리할 수 있습니다.

11. 책에서 구현 예시를 읽을 때 집중할 함수

함수	재혁님이 볼 포인트
`mm_init`	초기 heap 모양을 어떻게 만드는지, prologue/epilogue를 왜 두는지 보기
`extend_heap`	free block이 모자랄 때 heap을 늘리고 새 free block으로 바꾸는 흐름 보기
`find_fit`	free list를 어떻게 순회하며 맞는 block을 찾는지 보기
`place`	찾은 free block을 그대로 줄지, split할지 결정하는 부분 보기
`coalesce`	양옆 block 상태에 따라 4가지 경우로 합치는 부분 보기

12. 9.9를 읽을 때 추천 순서

1. **9.9.1 ~ 9.9.3**: heap, malloc/free, 왜 동적 할당이 필요한지, allocator 제약 읽기
2. **단편화**: internal / external fragmentation 차이 정리하기
3. **9.9.6 ~ 9.9.8**: implicit free list, placement, splitting 읽기
4. **9.9.11**: boundary tag와 coalescing 읽기
5. **9.9.12**: 구현 예시를 함수 흐름 위주로 읽기

읽으면서 계속 던질 질문 3개

- 이 함수는 heap을 어떻게 바꾸는가?
- 이 동작은 split인지, coalesce인지, search인지?
- 이 코드가 줄이려는 것은 internal fragmentation인가, external fragmentation인가?

13. 읽기 전에 이 정도만 기억하면 충분합니다

heap은 block들의 집합이고, allocator는 그 block들에 대해 **찾고(find), 나누고(split), 합치고(coalesce), 필요하면 늘리는(extend)** 관리자라고 생각하면 됩니다.

- `malloc`: 맞는 free block 찾기
- `free`: free로 바꾸고 합칠 수 있으면 합치기
- `realloc`: 크기 조정, 필요하면 새로 할당 후 복사
- **핵심 적**: 단편화
- **핵심 도구**: header/footer, implicit free list, split, coalesce

재혁님에게 가장 중요한 읽기 태도

이번에는 문장을 전부 이해하려고 하기보다, **heap 그림이 머릿속에 그려지는지**를 기준으로 읽으면 좋습니다. 코드 한 줄보다 block이 어떻게 변하는지에 집중하면 9.9가 훨씬 덜 어렵습니다.